

Lab Report 2

Course Name: Numerical Methods Lab

Submitted To: Mr. Kazi Abrar Mahmud

Submitted By: Md. Hussain Shahriar Rokon

Student Id: 0212330092

Date of Submission: 22 July, 2026

Task 1: False Position

Part 1: Matlab Implementation of False Position Method

The False Position Method developed upon Bisection method by incorporating linear interpolation between brackets $[x_l, x_u]$. The formula is given by:

$$x_r = x_u - \frac{f(x_u)(x_l - x_u)}{f(x_l) - f(x_u)}$$

Code:

```
% False position method
clc; clear; close all;
% defining function
f = @(x)x^3+x-1;

% inputs
xl = 0;
xu = 1;
tol = 0.001; %correct up to 3 decimal
max_iter = 100;

if f(xl)*f(xu) > 0
    error('No root exists in the given interval[xl,xu].');
end

xr = xl;
iter = 0;
ae = 100;%approximate error

fprintf('Iter\t xl\t\t\t xu\t\t\t xr\t\t\t ae(%%)\n');

while ae > tol && iter < max_iter
    iter = iter + 1;
    xr_old = xr;

    % False Position Formula
    xr = xu-(f(xu)*(xl - xu))/(f(xl)-f(xu));

    if xr ~=0
        ae=abs((xr-xr_old)/xr)*100;
    end

    fprintf('%d\t\t %.6f\t %.6f\t %.6f\t %.6f\n', iter, xl, xu, xr, ae);

    % updating brackets
    if f(xl)*f(xr)<0
        xu=xr;
    elseif f(xl)*f(xr)>0
        xl=xr;
    end
end
```

```

        else
            break;
        end
    end

fprintf('\nEstimated Root: %.4f\n', xr);

```

Output:

The screenshot shows the MATLAB Editor with a script named 'falsePosition1.m'. The script implements the False Position method to find the root of the function $f(x) = x^3 - x - 1$ within the interval $[0, 1]$. It sets a tolerance of 0.001 and a maximum of 100 iterations. The Command Window displays the iterative results in a table and the final estimated root.

Iter	xl	xu	xr	ae(%)
1	0.000000	1.000000	0.500000	100.000000
2	0.500000	1.000000	0.636364	21.428571
3	0.636364	1.000000	0.671196	5.189547
4	0.671196	1.000000	0.679662	1.245619
5	0.679662	1.000000	0.681691	0.297697
6	0.681691	1.000000	0.682176	0.071066
7	0.682176	1.000000	0.682292	0.016960
8	0.682292	1.000000	0.682319	0.004047
9	0.682319	1.000000	0.682326	0.000966

Estimated Root: 0.6823

Part 2: Stopping Criteria

Stopping criteria tell the program when to stop to avoid unnecessary calculations or infinite loops:

1. Absolute Relative Approximate Error (a_e):

$$a_e = \left| \frac{x_r^{\text{new}} - x_r^{\text{old}}}{x_r^{\text{new}}} \right| \times 100\% < \text{TOL}$$

2. Absolute Function Value Tolerance:

$$|f(x_r)| < \text{TOL}$$

Checks if plugging the root estimate into $f(x)$ gives a result close to zero.

3. Maximum Iteration Limit:

```
iter >= max_iter
```

Limits total loop count to prevent infinite execution if convergence fails.

Part 3: Improving the Method

One flaw with basic False Position is that curved graphs can cause one boundary point to get "**stuck**", slowing down the search.

Fix: If an endpoint doesn't move for two steps in a row, the algorithm cuts its function value in half ($f(x_{\text{stagnant}})/2$). This tilts the next interpolating line across the root, unsticking the boundary and speeding up convergence.

Part 4: Comparison Task (Bisection vs. False Position)

Evaluating $f(x) = 3x + \sin(x) - e^x = 0$ over 5 iterations in interval $[0, 1]$

Code:

```
clc; clear;
% define target equation and reference root
f = @(x) 3*x + sin(x) - exp(x);
exact_root = 0.360421703;
% initial brackets [0, 1] for both methods
xl_b = 0; xu_b = 1; % bisection value
xl_f = 0;
xu_f = 1; % false position value
% print table header
fprintf('Iter\tBisection x\tBisection f(x)\tFalse Pos x\tFalse Pos f(x)\n');
for i = 1:5
    % --- bisection method
    xr_b = (xl_b + xu_b) / 2;
    fx_b = f(xr_b);
    if f(xl_b)*fx_b < 0
        xu_b = xr_b;
    else
        xl_b = xr_b;
    end
    % --- false position method
    xr_f = xu_f - (f(xu_f) * (xl_f - xu_f)) / (f(xl_f) - f(xu_f));
    fx_f = f(xr_f);
    if f(xl_f)*fx_f < 0
        xu_f = xr_f;
    else
        xl_f = xr_f;
    end
    % print results for current step
    fprintf('%d\t%.6f\t%.6f\t%.6f\t%.6f\n', i, xr_b, fx_b, xr_f, fx_f);
end
```

```

1 clc; clear;
2
3 % define target equation and reference root
4 f = @(x) 3*x + sin(x) - exp(x);
5 exact_root = 0.360421703;
6
7 % initial brackets [0, 1] for both methods
8 xl_b = 0; xu_b = 1; % bisection value
9 xl_f = 0;
10 xu_f = 1; % false position value
11
12 % print table header
13 fprintf('Iter\tBisection x\tBisection f(x)\tFalse Pos x\tFalse Pos f(x)\n');
14
15 for i = 1:5
16     % --- bisection method
17     xr_b = (xl_b + xu_b) / 2;
18     fx_b = f(xr_b);
19
20     if f(xl_b)*fx_b < 0
21         xu_b = xr_b;
22     else
23         xl_b = xr_b;
24     end
25
26     % --- false position method
27     xr_f = xu_f - (f(xu_f) * (xl_f - xu_f)) / (f(xl_f) - f(xu_f));
28     fx_f = f(xr_f);
29
30     if f(xl_f)*fx_f < 0
31         xu_f = xr_f;
32     end
33 end

```

Iter	Bisection x	Bisection f(x)	False Pos x	False Pos f(x)
1	0.500000	0.330704	0.470990	0.265159
2	0.250000	-0.286621	0.372277	0.029534
3	0.375000	0.036281	0.361598	0.002941
4	0.312500	-0.121899	0.360537	0.000289
5	0.343750	-0.041956	0.360433	0.000028

Task 2: Open & Fixed Point

Target Non-linear Equation: $e^{(x^2)} + 0.3 \ln(x) = 2 \sqrt{x}$

1. MATLAB Plotting & Root Identification

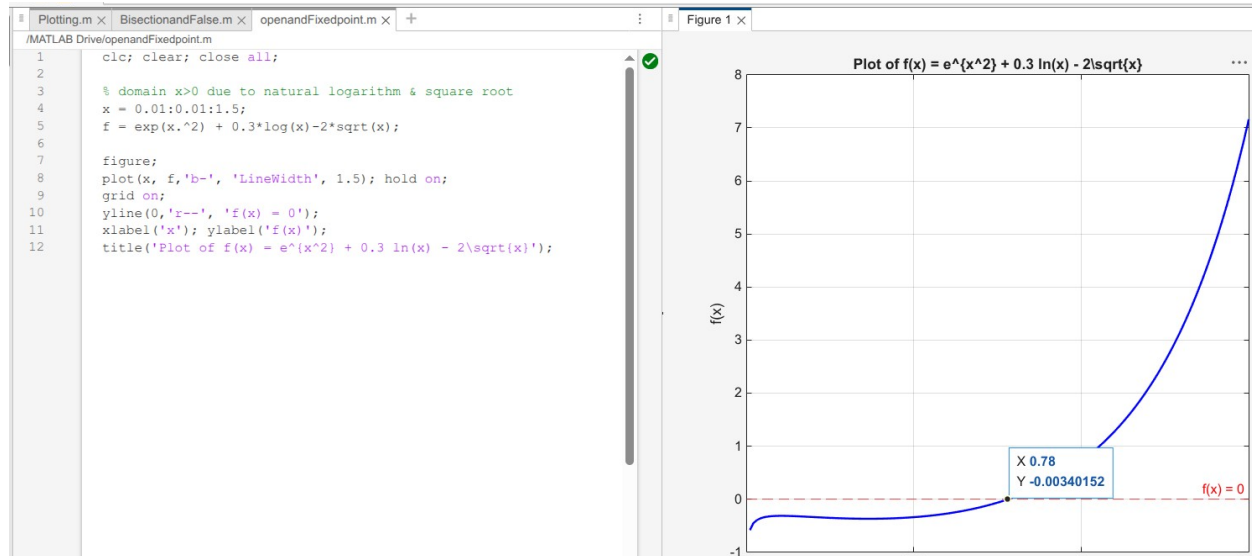
Code:

```

clc; clear; close all;

% domain x>0 due to natural logarithm & square root
x = 0.01:0.01:1.5;
f = exp(x.^2) + 0.3*log(x)-2*sqrt(x);
figure;
plot(x, f, 'b-', 'LineWidth', 1.5); hold on;
grid on;
yline(0, 'r--', 'f(x) = 0');
xlabel('x'); ylabel('f(x)');
title('Plot of f(x) = e^{x^2} + 0.3 ln(x) - 2\sqrt{x}');

```



Observation: The graph crosses the line $f(x) = 0$ at $x \sim 0.78$, indicating one real root between 0 and 1.

2. Rearrangement & Convergence Test

To set up fixed-point iteration $x_{(i+1)} = g(x_i)$ from $e^{x^2} + 0.3 \ln(x) = 2\sqrt{x}$:

- **Rearrangement 1 (isolating $\ln(x)$):**
 $g_1(x) = \exp([2\sqrt{x} - e^{x^2}] / 0.3)$
- **Rearrangement 2 (isolating $\sqrt{x}$):**
 $g_2(x) = [(e^{x^2} + 0.3 \ln(x)) / 2]^2$

Convergence Check ($|g'(x)| < 1$):

Evaluating derivatives near $x \approx 0.78$:

- $|g_1'(0.78)| \approx 4.51 > 1 \rightarrow$ Diverges
- $|g_2'(0.78)| < 1 \rightarrow$ Converges

So, $g_2(x)$ is the correct choice.

3. Validity of Initial Guess $x_0 = -1$

Invalid.

Both $\ln(x)$ and $\sqrt{x}$ require $x > 0$ in the real domain. Plugging in $x_0 = -1$ gives complex numbers, making it unusable as a real starting point.

4. Two Steps of Fixed-Point Iteration

Using $g(x) = [(e^{x^2} + 0.3 \ln(x)) / 2]^2$ and starting guess $x_0 = 0.5$:

- **Step 1 (i = 1):**

$$x_1 = g(0.5) = [(e^{(0.25)} + 0.3 \ln(0.5)) / 2]^2 = [(1.284 - 0.208) / 2]^2 = \mathbf{0.2895}$$

$$e_a = | (0.2895 - 0.5) / 0.2895 | * 100\% = \mathbf{72.7\%}$$

- **Step 2 (i = 2):**

$$x_2 = g(0.2895) = [(e^{(0.2895^2)} + 0.3 \ln(0.2895)) / 2]^2 = [(1.087 - 0.372) / 2]^2 = \mathbf{0.1280}$$

$$e_a = | (0.1280 - 0.2895) / 0.1280 | * 100\% = \mathbf{126.2\%}$$

Iterations continue until error drops below 10^{-7} as it approaches $x \approx 0.78$.

Task 3: Newton-Raphson Method & Pitfall Evaluation

Code:

```
clc; clear; close all;
%function with slope variation
f = @(x) x.^3 -2*x + 2;
df = @(x) 3*x.^2 -2;
% initial guess near a slope transition point
x0 = 0;
max_iter= 10;
x= x0;
fprintf('Iteration history for NewtonRaphson pitfall:\n');
for i=1:max_iter
%safety check for zero derivative
if df(x) == 0
error('derivative is zero. NewtonRaphson method fails.');
```

```
end
```

```
x_next=x- f(x)/df(x);
```

```
fprintf('iter %d:x= %.6f\n',i, x_next);
```

```
x =x_next;
```

```
end
```

Result:

Plotting.m xBisectionandFalse.m xopenandFixedpoint.m xNewtonRapson.m xNew.m x+

/MATLAB Drive/NewtonRapson.m

```
1 clc; clear; close all;
2
3 %function with slope variation
4 f = @(x) x.^3 -2*x + 2;
5 df = @(x) 3*x.^2 -2;
6 % initial guess near a slope transition point
7 x0 = 0;
8 max_iter= 10;
9 x= x0;
10
11 fprintf('Iteration history for NewtonRaphson pitfall:\n');
12
13 for i=1:max_iter
14     %safety check for zero derivative
15     if df(x) == 0
16         error('derivative is zero. NewtonRaphson method fails.');
```

Command Window

Iteration history for NewtonRaphson pitfall:
iter 1:x= 1.000000
iter 2:x= 0.000000
iter 3:x= 1.000000
iter 4:x= 0.000000
iter 5:x= 1.000000
iter 6:x= 0.000000
iter 7:x= 1.000000
iter 8:x= 0.000000
iter 9:x= 1.000000
iter 10:x= 0.000000
>>